

Task 1

Task Name

Create a Student Class with Encapsulation

Objective:

1. To understand the concept of encapsulation in Java.
2. To learn how to hide data using private access modifiers.
3. To use getter and setter methods for controlled access to data.
4. To improve data security and maintainability of a program.

Problem Analysis:

Encapsulation is one of the fundamental concepts of Object-Oriented Programming (OOP). It is the process of wrapping data and methods into a single unit (class) while restricting direct access to the data. In this task, a Student class is created with private data members: name, id, and cgpa. Setter methods are used to assign values, and getter methods are used to retrieve and display the values. This ensures that the data remains secure and can only be accessed through controlled methods.

Input:

- Student Name
- Student ID
- Student CGPA

Process:

- Create a Student class.
- Declare all data members as private.
- Create setter methods to assign values.
- Create getter methods to retrieve values.
- Create an object of the Student class.
- Set values using setter methods.
- Display values using getter methods.

Output:

The program displays the student's name, ID, and CGPA.

Algorithm

1. Start the program.
2. Create a class named Student.
3. Declare private variables name, id, and cgpa.
4. Create setter methods for all variables.
5. Create getter methods for all variables.
6. Create the main method.
7. Create an object of the Student class.
8. Set values using setter methods.
9. Display values using getter methods.
10. End the program.

Source Code:

```
class Student {  
    private String name;  
    private String id;  
    private double cgpa;  
    public void setName(String name) {  
        this.name = name;  
    }  
    public void setId(String id) {  
        this.id = id;  
    }  
    public void setCgpa(double cgpa) {  
        this.cgpa = cgpa;  
    }  
    public String getName() {  
        return name;  
    }  
}
```

```

    }

    public String getId() {
        return id;
    }

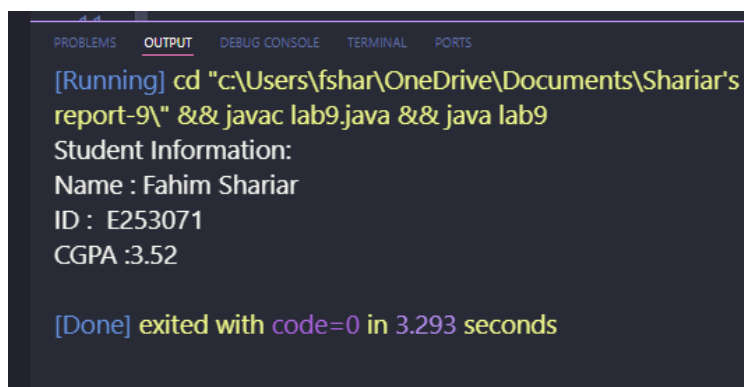
    public double getCgpa() {
        return cgpa;
    }
}

public class lab9 {

    public static void main(String[] args) {
        Student s = new Student();
        s.setName(" Fahim Shariar");
        s.setId(" E253071");
        s.setCgpa( 3.52);
        System.out.println( "Student Information:");
        System.out.println("Name :"+ s.getName());
        System.out.println( "ID : "+ s.getId());
        System.out.println("CGPA :"+ s.getCgpa());
    }
}

```

Output:



```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
[Running] cd "c:\Users\fshtar\OneDrive\Documents\Shariar's
report-9\" && javac lab9.java && java lab9
Student Information:
Name : Fahim Shariar
ID : E253071
CGPA :3.52

[Done] exited with code=0 in 3.293 seconds

```

Discussion

This program demonstrates the concept of encapsulation in Java. The data members are declared as private, preventing direct access from outside the class. Getter and setter methods provide controlled access to the data. This approach improves security, flexibility, and maintainability of the program.

Task 2:

Task Name:

Create a BankAccount Class with Encapsulation

Objective:

1. To implement encapsulation in a real-life banking system.
2. To protect sensitive account information using private variables.
3. To access and modify account information through getter and setter methods.
4. To understand secure data handling in object-oriented programming.

Problem Analysis:

In a banking system, account information should not be accessed directly by users. Encapsulation helps secure sensitive information by making data members private and providing public methods to access them. In this task, a BankAccount class is created with private fields: accountHolder, accountNumber, and balance. Setter methods are used to assign values and getter methods are used to retrieve and display account information securely.

Input:

- Account Holder Name
- Account Number
- Account Balance

Process:

- Create a BankAccount class.
- Declare all fields as private.

- Create setter methods to assign values.
- Create getter methods to retrieve values.
- Create an object of the BankAccount class.
- Set values through setter methods.
- Display account information using getter methods.

Output:

The program displays account holder information securely.

Algorithm

1. Start the program.
2. Create a class named BankAccount.
3. Declare private variables accountHolder, accountNumber, and balance.
4. Create setter methods for all variables.
5. Create getter methods for all variables.
6. Create the main method.
7. Create an object of the BankAccount class.
8. Set account information using setter methods.
9. Retrieve and display values using getter methods.
10. End the program.

Source Code:

```
class BankAccount {
    private String accountHolder ;
    private int accountNumber;
    private double balance;
    public void setAccountHolder(String accountHolder){
        this.accountHolder = accountHolder;
    }
    public void setAccountNumber(int accountNumber){
```

```

        this.accountNumber = accountNumber;
    }

    public void setBalance(double balance){
        this.balance = balance;
    }

    public String getAccountHolder(){
        return accountHolder;
    }

    public int getAccountNumber(){
        return accountNumber;
    }

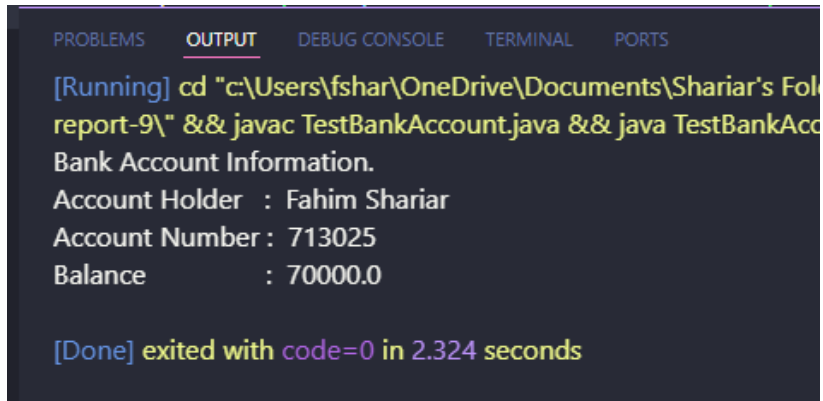
    public double getBalance(){
        return balance;
    }
}

public class TestBankAccount {
    public static void main( String[] args){
        BankAccount account = new BankAccount();
        account.setAccountHolder("Fahim Shariar");
        account.setAccountNumber(713025);
        account.setBalance(70000.00);

        System.out.println("Bank Account Information.");
        System.out.println("Account Holder: " + account.getAccountHolder());
        System.out.println("Account Number: " + account.getAccountNumber());
        System.out.println("Balance: " + account.getBalance());
    }
}

```

Output:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  
[Running] cd "c:\Users\fsnar\OneDrive\Documents\Shariar's Fol  
report-9\" && javac TestBankAccount.java && java TestBankAcc  
Bank Account Information.  
Account Holder : Fahim Shariar  
Account Number : 713025  
Balance : 70000.0  
[Done] exited with code=0 in 2.324 seconds
```

Discussion

This program illustrates the practical use of encapsulation in a banking application. The account information is stored in private variables and cannot be accessed directly from outside the class. Getter and setter methods provide secure and controlled access to the data. This technique enhances security, protects sensitive information, and follows good object-oriented programming practices.